

`AllowOverride None` prevents Apache from looking for `.htaccess` in each directory that it visits. For example, if the directive was set to `AllowOverride All` then if you were to request for `http://www.mydomain.com/member/address`, Apache will attempt to open a `.htaccess` file in `/htaccess`, `/var/htaccess`, and `/var/www/htaccess`. These lookups add to the latency. Setting the directive to 'None' improves performance because these lookups are not performed.

Set `Options FollowSymLinks` and not `Options SymLinksIfOwnerMatch`. The former directive will follow symbolic links in this directory. In case of the latter, symbolic links are followed only if the ownership of the link and the target match. To determine the ownership, Apache has to issue additional system calls. These additional calls add to the latency.

Avoid content negotiation altogether. Do not set `Options Multiviews`. If this is set and if the Apache server receives a request for a file that does not exist, it looks for files that closely match the requested file. It then chooses the best match from the available list and returns the document. This is rather unnecessary in most situations. It's better to trigger a 404.

In addition to this, you will need to set the following parameters in the `httpd.conf` (Centos) / `apache.conf` (Ubuntu) file depending on the distribution you are using.

MaxClients: This is the maximum concurrent requests that are supported by Apache. Setting this to 'Too low' will cause requests to queue up and eventually time-out, while server resources are left under utilised. Setting this to 'Very high' could cause your server to swap heavily and the response time degrades drastically. To calculate the MaxClients, take the maximum memory allocated to Apache and divide it by the maximum child process size. The child process size for a Drupal installation is anywhere between 30 to 40 MB compared to a ground-up PHP site, which is about 15 MB. The size might be different on your server depending on the number of modules. Use

```
ps -ylC httpd -sort:rss (on Centos) or
ps -ylC apache2 -sort:rss (on Ubuntu)
```

...to figure out the non-swapped physical memory usage by Apache (in KB). On our servers, with 2 GB of RAM, the MaxClients value is $(2 \times 1024) / 40$, i.e., 51. Apache has a default setting of 256 and you might want to revise it downward because the entire 2 GB is not allocated to Apache.

MinSpareServers, MaxSpareServers and StartServers: The MinSpareServers and MaxSpareServers determine how many child processes to keep active while Apache is waiting for requests. Set the MinSpareServers very low and a bunch of requests will come in; Apache has to spawn additional child processes to serve these requests. Creating child processes takes time and if Apache is busy creating these processes, the client requests won't be served immediately. If the MaxSpareServers is set

too high then unnecessary resources are consumed while Apache listens for requests. Set MinSpareServers to 5 and MaxSpareServers to 20.

StartServers specifies the number of child server processes that are created on start-up. On start-up, Apache will continue to create child processes till the MinSpareServers limit is reached. This doesn't have an effect on performance when the Apache server is running. However, if you restart Apache frequently (seldom would anyone do so) and there are a lot of requests to be serviced, then set this to a relatively high value.

MaxRequestsPerChild: The MaxRequestsPerChild directive sets the limit on the number of requests that an individual child server process will handle before it dies. By default, it is set to 0. Set this to a finite value—usually a few thousands. Our servers have it set to 4000. It helps prevent memory leakage.

KeepAlive: KeepAlive allows multiple requests to be sent over the same TCP connection. Turn this on if you have a site with a lot of images—otherwise for each image a separate TCP connection is established and establishing a new connection has its overheads, adding to the delay.

KeepAliveTimeout: This specifies how long Apache has to wait for the next request before timing out the connection. Set this to 2 - 5 seconds. Setting this to a high value could tie up connections that wait for requests when they can be used to serve new clients.

Disable Logging in access.log. If you would like to go to the extremes, you can disable Apache from logging each request it receives by commenting the CustomLog directive in `httpd.conf`.

A similar directive is likely to be in your `vhost` configuration so ensure that you comment it out. I wouldn't recommend it though, because access logs are used to provide an insight into visitor behaviour. Instead, install 'log rotate'.

On Ubuntu, use:

```
sudo apt-get install logrotate
```

On Centos, however, `log rotate` is run daily via the file `/etc/cron.daily/logrotate`. The configuration files for it are available in `/etc/logrotate.conf` and `/etc/logrotate.d`

Increase the Apache buffer size. For very high latency networks, especially transcontinental connections, you can set the TCP buffer size in Apache with the directive `SendBufferSize` bytes. 

By: Sridhar Pandurangiah

The author is the co-founder and director of Sastra Technologies, a start-up engaged in providing EDI solutions on the cloud. He can be contacted at: sridhar@sastratechologies.in / sridharpandu@gmail.com. He maintains a technical blog: sridharpandu.wordpress.com